

DVSA

Vulnerability Discovery & Remediation

Course Project — ICS-344 Information Security
King Fahd University of Petroleum and Minerals
Term 252 | Team 54

Aseel Bawazir 202270080

Mohammed AlZaid 202272600

Mohammed Alsuhaibani 202245800

10

Official
Vulnerabilities

AWS

Serverless
Platform

100%

Exploited & Fixed

Presentation Agenda

01

Setup & Architecture

DVSA deployment on AWS Serverless

02

Lessons 1 & 9

Event Injection + Vulnerable Dependencies

03

Lesson 2

Broken Authentication (JWT Forgery)

04

Lessons 3 & 4

Sensitive Data Disclosure + Insecure Cloud Config

05

Lesson 5

Broken Access Control

06

Lessons 6, 7 & 8

DoS + Over-Privileged Functions + Race Condition

07

Lesson 10

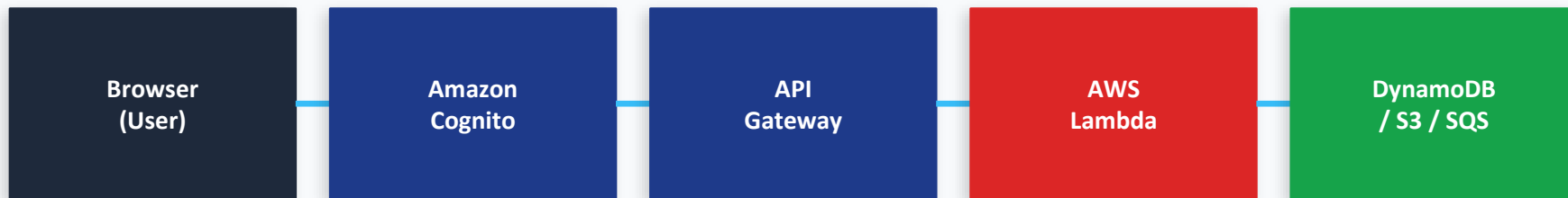
Unhandled Exceptions

08

Summary & Takeaways

Key security lessons learned

DVSA — AWS Serverless Architecture



⚠️ ATTACK SURFACE — Vulnerabilities exist at every layer

Frontend

Static site on S3 / CloudFront

Backend

API Gateway + Lambda functions

Data

DynamoDB · S3 Buckets · SQS

Event Injection & Vulnerable Dependencies

Root vector: node-serialize library enables Remote Code Execution via attacker-controlled JSON payloads sent through API Gateway.

RCE

Remote Code Execution

0

Auth Required

CVE

Vulnerable Dependency

Event Injection + Vulnerable Dependencies



How It Works

The DVSA-ORDER-MANAGER Lambda uses node-serialize v0.0.4 to parse the HTTP request body. This library evaluates embedded JavaScript functions when it encounters the special `__$ND_FUNC__$` prefix in JSON input — effectively calling `eval()` on attacker-supplied data before any authentication occurs.



Exploit Path

Attacker sends a POST to `/order` with a crafted JSON payload containing `__$ND_FUNC__$_function(){...}()`. The Lambda deserializes it, executes the injected code, and logs FILE READ SUCCESS to CloudWatch — even though the API returns a generic 500 error to the client.



Fix Applied

Remove node-serialize from package.json entirely. Replace `serialize.unserialize()` with `JSON.parse()` wrapped in try/catch. Add an explicit allowlist validation for the action field (new, get, orders, shipping, billing, cancel) before any further processing.



Security Impact

Full backend code execution with zero authentication. Attacker can read `/tmp` files, dump Lambda environment variables (including AWS credentials), and interact with all AWS services using the function's IAM role.

Broken Authentication (JWT Forgery)

The backend decodes JWT payloads without verifying the cryptographic signature, allowing any authenticated user to impersonate any other user.

Header

Payload

Signature

Broken Authentication — JWT Forgery



How It Works

The Lambda handler splits the Authorization header token on '.', base64-decodes the middle section (the payload), and reads username and sub directly from the resulting JSON — without ever calling any signature verification function. The signature is completely ignored.



Exploit Path

User B logs in and captures their TOKEN_B from DevTools. They decode the payload, replace username and sub with User C's identity (VICTIM_USER), re-encode the payload, and reattach the original signature. Sending this forged token returns User C's full order list.



Fix Applied

Add `verifyCognitoJwt()` helper that fetches Cognito JWKS public keys and calls `jose.JWS.createVerify()` to cryptographically validate the token. Validate iss and exp claims. Only then extract user identity from claims. Forged tokens now return HTTP 401.



Security Impact

Complete account impersonation: attacker retrieves any victim's order list, order details, shipping address, and total amounts. Scales to any user in the system — attacker only needs a valid token of their own plus any victim identifier.

Sensitive Data Disclosure & Insecure Cloud Config

Lesson 3: Receipt function generates presigned S3 URLs without ownership checks.

Lesson 4: S3 bucket accepts unauthenticated writes, triggering Lambda via S3 events.

Lesson 3

Missing ownership
check on receipts

Lesson 4

Public S3 bucket
+ Lambda trigger

Sensitive Data Disclosure + Insecure Cloud Configuration



How It Works

Lesson 3: DVSA-ADMIN-GET-RECEIPT generates a presigned S3 URL for any order-id without verifying that the caller owns that order — it never queries DynamoDB to compare the order's userId with the authenticated JWT identity.

Lesson 4: The receipts S3 bucket has no Block Public Access settings and a permissive policy, allowing unauthenticated writes that trigger the receipt Lambda via S3 events.



Exploit Path

L3: Attacker uses Lesson 2 exploit to get victim's order-id, then calls the receipt endpoint with their own token — receives a presigned URL for victim's receipt and downloads it.

L4: Attacker runs: `aws s3 cp file.txt s3://[BUCKET]/path --no-sign-request`. Upload succeeds and triggers DVSA-SEND-RECEIPT-EMAIL Lambda with attacker-controlled data.



Fix Applied

L3: Add DynamoDB ownership check before URL generation — compare order's userId with JWT sub. Return 403 if mismatch. Reduce presigned URL expiry to 5 minutes.

L4: Enable all 4 S3 Block Public Access settings. Replace permissive bucket policy with least-privilege policy scoped to authorized Lambda role ARNs only.



Security Impact

L3: Any authenticated user can download any other user's private financial receipt documents.

L4: Any internet client can write arbitrary files to the receipts bucket, trigger Lambda functions with attacker-controlled content, and potentially pivot to further exploitation.

Broken Access Control (Payment Bypass)

Admin update function is reachable from the public billing API path without verifying payment completion, allowing any order to be marked paid for \$0.

A rectangular button with a light blue background and a thin red border. The word "NEW" is centered in white, uppercase letters.

NEW

A rectangular button with a solid orange background. The word "SHIPPED" is centered in white, uppercase letters.

SHIPPED

A rectangular button with a solid red background and a thin green border. The word "PAID" is centered in white, uppercase letters.

PAID

Broken Access Control — Payment Bypass



How It Works

The application is designed so orders progress new → shipped → paid only after real payment. However, DVSA-ADMIN-UPDATE-ORDERS (which writes 'paid' status to DynamoDB) can be reached from the public billing API path without verifying that actual payment processing succeeded. No server-side state transition validation exists.



Exploit Path

Attacker places an order and advances it to shipped state. They then send a billing request to the /order API endpoint which internally invokes DVSA-ADMIN-UPDATE-ORDERS. The admin function marks the order as 'paid' with no real payment charged. Total cost: \$0.



Fix Applied

Add DynamoDB state validation in billing Lambda: check order must be in 'shipped' state before invoking admin update. In DVSA-ADMIN-UPDATE-ORDERS: reject invocations arriving via API Gateway (event.requestContext.present). Only legitimate internal Lambda calls proceed.



Security Impact

Attacker receives goods/services for free by bypassing the entire payment system. In a real application this would represent direct financial fraud. All orders for any authenticated user can be marked as paid without payment.

DoS · Over-Privileged Functions · Race Condition

Three distinct attack classes targeting availability, IAM permissions, and business logic integrity.

Lesson 6 — DoS

50 concurrent requests saturate Lambda pool

Lesson 7 — Over-Priv

Wildcard IAM role exposes all DynamoDB tables

Lesson 8 — Race Cond

TOCTOU: 5 items billed at 1-item price

Lessons 6, 7 & 8 — DoS · Over-Privileged Functions · Race Condition

Lesson 6 — DoS

Root Cause

No rate limiting or Lambda reserved concurrency — 50 concurrent requests exhaust the shared pool

Exploit

Python threading.Barrier fires 50 simultaneous requests; victim gets 429 errors or 3x+ latency

Fix

API Gateway usage plan (2 req/s, burst 5) + Lambda reserved concurrency (10) on ORDER-MANAGER

Lesson 7 — Over-Privileged IAM

Root Cause

DVSA-SEND-RECEIPT-EMAIL role grants s3:* on all buckets and dynamodb:Scan on all tables via wildcards

Exploit

IAM Policy Simulator confirms ALLOWED for arbitrary S3/DynamoDB resources. Stolen STS credentials scan DVSA-ORDERS-DB

Fix

Detach AmazonSESFullAccess; apply inline least-privilege policy scoped to specific DVSA bucket + table ARNs

Lesson 8 — Race Condition

Root Cause

TOCTOU gap: billing reads quantity, charges price, then writes 'paid' — no atomic lock between read and write

Exploit

Concurrent billing (1 item, \$25) + update (qty=5) via threading.Barrier; final state: 5 items at \$25 total

Fix

DynamoDB ConditionExpression CAS lock: shipped → billing_pending atomically; update Lambda rejects locked orders

Unhandled Exceptions — Information Leakage

Lambda handlers propagate raw JavaScript exceptions to API Gateway, exposing internal stack traces, file paths, and code structure to any attacker.

EXCEPTION

HTTP 500

EXPOSED

Unhandled Exceptions — Information Leakage



How It Works

Lambda handler functions do not wrap body parsing or main logic in try/catch blocks. When malformed, incomplete, or unexpected input is received, JavaScript throws `TypeError` or `SyntaxError` exceptions that propagate directly to API Gateway. API Gateway returns the full error object — including stack traces and internal file paths — to the client.



Exploit Path

Attacker sends malformed requests: empty body {}, null action, numeric action field 12345, or invalid JSON. API response contains: Node.js stack traces with function names and line numbers, `/var/task/` internal file paths, `TypeError` messages revealing code structure.



Fix Applied

Add try/catch around `JSON.parse()` at the top of every handler — return generic 400 on parse failure. Add `safeError()` helper that logs full details to CloudWatch internally but returns only a generic client-safe message. Wrap all promise chains with `.catch()` calling `safeError()`.



Security Impact

Internal architecture reconnaissance: attacker learns file paths (`/var/task/order-manager.js`), function names, line numbers, Node.js version, and code structure. This information directly aids further targeted attacks and vulnerability chaining.

Vulnerability Summary — All 10 Lessons

Lesson	Vulnerability	Affected Component	Fix Summary
#1 + #9	Event Injection + Vuln. Deps	ORDER-MANAGER / node-serialize	Remove node-serialize; JSON.parse + allowlist
#2	Broken Authentication (JWT)	ORDER-MANAGER JWT handling	Verify JWT signature via Cognito JWKS
#3	Sensitive Data Disclosure	ADMIN-GET-RECEIPT function	DynamoDB ownership check before S3 URL
#4	Insecure Cloud Configuration	S3 Receipts Bucket	Enable Block Public Access + least-priv policy
#5	Broken Access Control	ADMIN-UPDATE-ORDERS function	State machine validation + caller check
#6	Denial of Service	API Gateway / Lambda pool	Usage plan rate limit + reserved concurrency
#7	Over-Privileged Functions	SEND-RECEIPT-EMAIL IAM Role	Least-privilege inline policy (specific ARNs)
#8	Logic Vulnerability (Race Cond.)	ORDER-MANAGER billing flow	DynamoDB ConditionExpression CAS lock
#10	Unhandled Exceptions	All Lambda handlers	safeError() helper + try/catch wrapping

Key Takeaways

Principles for securing AWS serverless applications



Never trust user input

Deserialize and validate strictly — treat every API request as potentially malicious



Enforce state server-side

State transitions, payment workflows, and access control must be validated at every function boundary



Verify, never just decode

JWT signatures must be cryptographically verified server-side using the IDP's public keys



Rate limit and isolate

Reserved concurrency + usage plans prevent DoS and protect other functions from being starved



Least privilege is mandatory

Lambda execution roles must be scoped to specific resources — wildcard ARNs create blast radius



Error handling = security

Never leak stack traces or paths to clients — log internally, return generic messages externally

Project Deliverables

- ✓ Written Report
- ✓ Presentation Slides
- ✓ GitHub Repository
- ✓ Demo Video Recording

ICS-344 | Term 252 | KFUPM